Technical Architecture Document

Intelligent Recruitment Matching Platform

---

1. System Overview

Architecture Pattern

Microservices-oriented monolith (Modular Monolith evolving to Microservices)

This approach allows for:

- Initial rapid development with a monolithic structure
- Clear module boundaries for future microservices migration
- Shared database initially, with clear schema separation
- Independent deployment capabilities as system matures

High-Level Architecture Layers

```
┌──────────────────────────────────────────────────────────┐
│                    Presentation Layer                    │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐   │
│  │  Web Client  │  │ Mobile App   │  │ BI Dashboard │   │
│  │  (React)     │  │  (Future)    │  │  (Tableau)   │   │
│  └──────────────┘  └──────────────┘  └──────────────┘   │
└──────────────────────────────────────────────────────────┘
                           │
                     ┌─────┴──────┐
                     │  API Gateway │
                     │  (Kong/Nginx)│
                     └────────────┘
                           │
┌──────────────────────────────────────────────────────────┐
│                   Application Layer                      │
│  ┌──────────────────────────────────────────────────┐   │
│  │           REST API (FastAPI)                     │   │
│  │  ┌────────┐ ┌────────┐ ┌────────┐ ┌──────────┐  │   │
│  │  │  Auth  │ │Candidate│ │  Jobs  │ │ Matching │  │   │
│  │  │ Module │ │ Module │ │ Module │ │  Module  │  │   │
│  │  └────────┘ └────────┘ └────────┘ └──────────┘  │   │
│  └──────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────┘
                           │
┌──────────────────────────────────────────────────────────┐
```

```
│                    Processing Layer                          │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐      │
│  │ ETL Pipeline │  │ ML Training  │  │  ML Serving  │      │
│  │  (Airflow)   │  │  (MLflow)    │  │   (FastAPI)  │      │
│  └──────────────┘  └──────────────┘  └──────────────┘      │
│  ┌──────────────┐  ┌──────────────┐                        │
│  │ NLP Engine   │  │  CV Engine   │                        │
│  │   (spaCy)    │  │  (OpenCV)    │                        │
│  └──────────────┘  └──────────────┘                        │
└──────────────────────────────────────────────────────────────┘
                              │
┌──────────────────────────────────────────────────────────────┐
│                      Data Layer                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐      │
│  │  PostgreSQL  │  │   MongoDB    │  │    Redis     │      │
│  │  (Relational)│  │  (Documents) │  │   (Cache)    │      │
│  └──────────────┘  └──────────────┘  └──────────────┘      │
│  ┌──────────────┐  ┌──────────────┐                        │
│  │ S3/MinIO     │  │ Elasticsearch│                        │
│  │ (Files/ML)   │  │ (Search)     │                        │
│  └──────────────┘  └──────────────┘                        │
└──────────────────────────────────────────────────────────────┘
```

---

2. Detailed Component Specifications

2.1 Data Ingestion Layer

Component: Data Collection Service

Technology: Python 3.10+, Pandas, Apache Airflow **Purpose**: Collect and validate incoming data from multiple sources

Data Sources:

1. **Job Postings**
   - Formats: JSON, CSV, XML
   - Sources: Manual upload, API integration, web scraping (with permission)
   - Volume: ~1,000-10,000 jobs/day
   - Schema validation: Pydantic models
2. **Candidate CVs**
   - Formats: PDF, DOCX, TXT
   - Max file size: 10MB
   - Storage: S3/MinIO → preprocessing → metadata in PostgreSQL

- OCR for scanned PDFs: Tesseract

3. **Video Interviews**
   - Formats: MP4, AVI, MOV, WebM
   - Max file size: 500MB
   - Frame rate: 24-30 FPS
   - Resolution: 720p minimum
   - Storage: S3 with lifecycle policies (delete after 90 days)

Validation Rules:

```python
# Example Pydantic schema for job posting validation
from pydantic import BaseModel, validator
from typing import List, Optional
from datetime import datetime

class JobPosting(BaseModel):
    title: str
    company: str
    description: str
    requirements: List[str]
    skills_required: List[str]
    experience_years: int
    salary_min: Optional[int]
    salary_max: Optional[int]
    location: str
    job_type: str  # full-time, part-time, contract
    category: str
    posted_date: datetime

    @validator('experience_years')
    def validate_experience(cls, v):
        if v < 0 or v > 50:
            raise ValueError('Invalid experience years')
        return v

    @validator('skills_required')
    def validate_skills(cls, v):
        if len(v) == 0:
            raise ValueError('At least one skill required')
        return [skill.lower().strip() for skill in v]
```

2.2 ETL Pipeline Layer

Component: Data Cleaning & Transformation

Technology: Pandas, Great Expectations, Apache Airflow **Schedule**: Hourly for incremental, daily for full refresh

Pipeline Stages:

Stage 1: Data Validation

```python
# Great Expectations validation suite
expectations = [
    {
        "expectation_type": "expect_column_values_to_not_be_null",
        "kwargs": {"column": "job_title"}
    },
    {
        "expectation_type": "expect_column_values_to_match_regex",
        "kwargs": {
            "column": "email",
            "regex": r"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$"
        }
    },
    {
        "expectation_type": "expect_column_values_to_be_in_set",
        "kwargs": {
            "column": "job_type",
            "value_set": ["full-time", "part-time", "contract", "internship"]
        }
    }
]
```

Stage 2: Data Cleaning

- Remove duplicates based on hash of key fields
- Handle missing values (imputation or removal)
- Standardize formats (dates, phone numbers, addresses)
- Remove HTML/special characters from text fields
- Normalize skill names using taxonomy mapping

Stage 3: Data Enrichment

- Geocoding for location data
- Industry classification

- Seniority level inference
- Skill taxonomy mapping

Airflow DAG Structure:

```python
```

```python
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'data-team',
    'depends_on_past': False,
    'start_date': datetime(2024, 1, 1),
    'email_on_failure': True,
    'email_on_retry': False,
    'retries': 3,
    'retry_delay': timedelta(minutes=5),
}

dag = DAG(
    'cv_processing_pipeline',
    default_args=default_args,
    description='Process and extract information from CVs',
    schedule_interval='@hourly',
    catchup=False
)

extract_task = PythonOperator(
    task_id='extract_raw_data',
    python_callable=extract_raw_cvs,
    dag=dag
)

validate_task = PythonOperator(
    task_id='validate_data',
    python_callable=validate_cv_data,
    dag=dag
)

parse_task = PythonOperator(
    task_id='parse_cvs',
    python_callable=parse_cv_content,
    dag=dag
)

feature_task = PythonOperator(
    task_id='extract_features',
    python_callable=extract_cv_features,
    dag=dag
)
```

```
load_task = PythonOperator(
    task_id='load_to_database',
    python_callable=load_parsed_data,
    dag=dag
)

extract_task >> validate_task >> parse_task >> feature_task >> load_task
```

2.3 NLP Processing Layer

Component: Resume Parser

Technology: spaCy 3.5+, transformers (Hugging Face), PyResParser **Models**: en_core_web_lg, BERT-based NER

Information Extraction Pipeline:

1. Text Extraction

```python
import pdfplumber
from docx import Document
import pytesseract
from PIL import Image

def extract_text_from_pdf(pdf_path: str) -> str:
    """Extract text from PDF, with OCR fallback for scanned documents"""
    try:
        with pdfplumber.open(pdf_path) as pdf:
            text = ""
            for page in pdf.pages:
                text += page.extract_text() or ""

            # If very little text extracted, assume scanned PDF
            if len(text.strip()) < 100:
                return extract_text_with_ocr(pdf_path)
            return text
    except Exception as e:
        logger.error(f"Error extracting text from {pdf_path}: {e}")
        return ""

def extract_text_from_docx(docx_path: str) -> str:
    """Extract text from DOCX"""
    doc = Document(docx_path)
    return "\n".join([paragraph.text for paragraph in doc.paragraphs])
```

2. Named Entity Recognition

```python
```

```python
import spacy
from spacy.matcher import Matcher
from typing import Dict, List

nlp = spacy.load("en_core_web_lg")

def extract_contact_info(text: str) -> Dict:
    """Extract email, phone, LinkedIn, etc."""
    doc = nlp(text)

    contact_info = {
        'emails': [],
        'phones': [],
        'linkedin': None,
        'github': None
    }

    # Email extraction
    email_pattern = r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b'
    contact_info['emails'] = re.findall(email_pattern, text)

    # Phone extraction (various formats)
    phone_patterns = [
        r'\+?1?\s*\(?([0-9]{3})\)?[-.\s]?([0-9]{3})[-.\s]?([0-9]{4})',
        r'\+[0-9]{1,3}\s?[0-9]{8,12}'
    ]
    for pattern in phone_patterns:
        phones = re.findall(pattern, text)
        contact_info['phones'].extend(phones)

    # LinkedIn URL
    linkedin_pattern = r'linkedin\.com/in/([a-zA-Z0-9-]+)'
    linkedin_match = re.search(linkedin_pattern, text)
    if linkedin_match:
        contact_info['linkedin'] = linkedin_match.group(0)

    return contact_info

def extract_education(text: str) -> List[Dict]:
    """Extract education information"""
    doc = nlp(text)

    # Common degree patterns
    degree_patterns = [
        "Bachelor", "Master", "PhD", "MBA", "B.S.", "M.S.",
        "B.A.", "M.A.", "B.Tech", "M.Tech"
```

```python
    ]

    education = []

    # Section-based extraction (look for "Education" section)
    education_section = extract_section(text, "education")

    if education_section:
        # Extract degrees, universities, years
        for sent in nlp(education_section).sents:
            sent_text = sent.text

            # Find degree
            degree = None
            for deg in degree_patterns:
                if deg.lower() in sent_text.lower():
                    degree = deg
                    break

            # Find year (4-digit number)
            years = re.findall(r'\b(19|20)\d{2}\b', sent_text)

            # Find university (organizations)
            orgs = [ent.text for ent in sent.ents if ent.label_ == "ORG"]

            if degree:
                education.append({
                    'degree': degree,
                    'institution': orgs[0] if orgs else None,
                    'year': years[-1] if years else None,
                    'field': extract_field_of_study(sent_text)
                })

    return education

def extract_work_experience(text: str) -> List[Dict]:
    """Extract work experience"""
    experience_section = extract_section(text, "experience")

    experiences = []

    if experience_section:
        # Split by common separators
        entries = re.split(r'\n(?=[A-Z].*[0-9]{4})', experience_section)

        for entry in entries:
            doc = nlp(entry)
```

```python
        # Extract job title (often at start)
        job_title = extract_job_title(entry)

        # Extract company (ORG entities)
        companies = [ent.text for ent in doc.ents if ent.label_ == "ORG"]

        # Extract dates
        dates = extract_date_range(entry)

        # Extract responsibilities (bullet points)
        responsibilities = extract_bullet_points(entry)

        experiences.append({
            'title': job_title,
            'company': companies[0] if companies else None,
            'start_date': dates.get('start'),
            'end_date': dates.get('end'),
            'responsibilities': responsibilities,
            'duration_months': calculate_duration(dates)
        })

    return experiences
```

3. Skill Extraction

```python
```

```python
from sklearn.feature_extraction.text import TfidfVectorizer
import json

# Load skill taxonomy
with open('skill_taxonomy.json', 'r') as f:
    SKILL_TAXONOMY = json.load(f)

def extract_skills(text: str) -> Dict[str, List[str]]:
    """Extract technical and soft skills"""
    text_lower = text.lower()

    skills = {
        'technical': [],
        'soft': [],
        'tools': [],
        'languages': []
    }

    # Technical skills (programming languages, frameworks, etc.)
    for skill_category, skill_list in SKILL_TAXONOMY.items():
        for skill in skill_list:
            # Use word boundaries to avoid partial matches
            pattern = r'\b' + re.escape(skill.lower()) + r'\b'
            if re.search(pattern, text_lower):
                skills[skill_category].append(skill)

    # Remove duplicates and sort
    for category in skills:
        skills[category] = sorted(list(set(skills[category])))

    return skills

# Skill taxonomy example structure
SKILL_TAXONOMY = {
    'technical': [
        'python', 'java', 'javascript', 'c++', 'sql', 'react', 'angular',
        'django', 'flask', 'tensorflow', 'pytorch', 'scikit-learn',
        'docker', 'kubernetes', 'aws', 'azure', 'gcp'
    ],
    'soft': [
        'leadership', 'communication', 'teamwork', 'problem solving',
        'project management', 'critical thinking'
    ],
    'tools': [
        'git', 'jira', 'confluence', 'tableau', 'power bi', 'excel'
    ],
```

```
    'languages': [
        'english', 'spanish', 'french', 'german', 'mandarin', 'arabic'
    ]
}
```

4. Semantic Embeddings

```python
from sentence_transformers import SentenceTransformer
import numpy as np

# Load pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

def generate_skill_embeddings(skills: List[str]) -> np.ndarray:
    """Generate embeddings for skills"""
    if not skills:
        return np.zeros(384)  # Default dimension for all-MiniLM-L6-v2

    # Generate embeddings for each skill
    skill_embeddings = model.encode(skills)

    # Average pooling
    avg_embedding = np.mean(skill_embeddings, axis=0)

    return avg_embedding

def generate_cv_embedding(cv_text: str) -> np.ndarray:
    """Generate embedding for entire CV"""
    # Use first 512 tokens (model limit)
    truncated_text = ' '.join(cv_text.split()[:512])
    embedding = model.encode(truncated_text)
    return embedding
```

2.4 Computer Vision Layer

Component: Video Emotion Detection

Technology: OpenCV, MediaPipe, TensorFlow/PyTorch, FER **Models**: FaceNet for face detection, ResNet for emotion classification

Video Processing Pipeline:

1. Frame Extraction

```python
import cv2
import numpy as np
from typing import List, Tuple

def extract_frames(video_path: str, fps: int = 1) -> List[np.ndarray]:
    """
    Extract frames from video at specified FPS
    Default: 1 frame per second for emotion analysis
    """
    cap = cv2.VideoCapture(video_path)

    video_fps = cap.get(cv2.CAP_PROP_FPS)
    frame_interval = int(video_fps / fps)

    frames = []
    frame_count = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        if frame_count % frame_interval == 0:
            frames.append(frame)

        frame_count += 1

    cap.release()
    return frames
```

2. Face Detection

```python
```

```python
import mediapipe as mp

mp_face_detection = mp.solutions.face_detection
mp_drawing = mp.solutions.drawing_utils

def detect_faces(frame: np.ndarray) -> List[Tuple[int, int, int, int]]:
    """
    Detect faces in frame using MediaPipe
    Returns list of bounding boxes (x, y, width, height)
    """
    with mp_face_detection.FaceDetection(
        model_selection=1,  # 1 for full-range model
        min_detection_confidence=0.5
    ) as face_detection:

        # Convert BGR to RGB
        rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

        results = face_detection.process(rgb_frame)

        faces = []
        if results.detections:
            for detection in results.detections:
                bboxC = detection.location_data.relative_bounding_box
                ih, iw, _ = frame.shape

                x = int(bboxC.xmin * iw)
                y = int(bboxC.ymin * ih)
                w = int(bboxC.width * iw)
                h = int(bboxC.height * ih)

                faces.append((x, y, w, h))

        return faces
```

3. Emotion Classification

```python
```

```python
from fer import FER
import torch
from torchvision import transforms

# Option 1: Using FER library (simpler)
emotion_detector = FER(mtcnn=True)

def detect_emotions_fer(frame: np.ndarray) -> Dict[str, float]:
    """Detect emotions using FER library"""
    result = emotion_detector.detect_emotions(frame)

    if result:
        emotions = result[0]['emotions']
        return emotions
    return {}

# Option 2: Custom deep learning model (more control)
class EmotionClassifier:
    def __init__(self, model_path: str):
        self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
        self.model = self.load_model(model_path)
        self.model.eval()

        self.transform = transforms.Compose([
            transforms.ToPILImage(),
            transforms.Resize((224, 224)),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406],
                                 std=[0.229, 0.224, 0.225])
        ])

        self.emotion_labels = ['angry', 'disgust', 'fear', 'happy',
                               'neutral', 'sad', 'surprise']

    def predict(self, face_image: np.ndarray) -> Dict[str, float]:
        """Predict emotion probabilities"""
        # Preprocess
        input_tensor = self.transform(face_image).unsqueeze(0).to(self.device)

        # Predict
        with torch.no_grad():
            outputs = self.model(input_tensor)
            probabilities = torch.nn.functional.softmax(outputs, dim=1)[0]

        # Create emotion dictionary
        emotions = {
```

```
            label: float(prob)
            for label, prob in zip(self.emotion_labels, probabilities)
        }

        return emotions
```

4. Emotion Feature Engineering

```python
```

```python
from scipy import stats
import pandas as pd

def analyze_emotion_timeline(video_emotions: List[Dict[str, float]]) -> Dict:
    """
    Analyze emotion patterns over time
    Returns aggregated features for ML model
    """
    if not video_emotions:
        return {}

    # Convert to DataFrame for easier analysis
    df = pd.DataFrame(video_emotions)

    features = {}

    # For each emotion, calculate statistics
    for emotion in df.columns:
        emotion_values = df[emotion].values

        features[f'{emotion}_mean'] = np.mean(emotion_values)
        features[f'{emotion}_std'] = np.std(emotion_values)
        features[f'{emotion}_max'] = np.max(emotion_values)
        features[f'{emotion}_min'] = np.min(emotion_values)

        # Trend analysis (is emotion increasing/decreasing?)
        if len(emotion_values) > 2:
            slope, _, _, _, _ = stats.linregress(
                range(len(emotion_values)),
                emotion_values
            )
            features[f'{emotion}_trend'] = slope

    # Dominant emotion (overall)
    mean_emotions = df.mean()
    features['dominant_emotion'] = mean_emotions.idxmax()
    features['dominant_emotion_score'] = mean_emotions.max()

    # Emotion variability (how stable are emotions?)
    features['emotion_variability'] = df.std().mean()

    # Positive vs Negative emotion ratio
    positive_emotions = ['happy', 'surprise']
    negative_emotions = ['angry', 'disgust', 'fear', 'sad']

    positive_score = df[positive_emotions].mean().sum()
```

```python
    negative_score = df[negative_emotions].mean().sum()

    features['positive_negative_ratio'] = (
        positive_score / negative_score if negative_score > 0 else 0
    )

    # Confidence level (average probability of dominant emotion)
    features['confidence_level'] = df.max(axis=1).mean()

    return features
```

2.5 Machine Learning Layer

Component: Job Classification Model

Technology: scikit-learn, XGBoost, MLflow **Model Type**: Multi-class classification

Feature Engineering:

```python
```

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import StandardScaler
import pandas as pd

def prepare_job_classification_features(job_data: pd.DataFrame) -> pd.DataFrame:
    """Prepare features for job category classification"""

    # Text features: TF-IDF on job description
    tfidf = TfidfVectorizer(
        max_features=1000,
        ngram_range=(1, 2),
        stop_words='english'
    )

    tfidf_features = tfidf.fit_transform(job_data['description'])
    tfidf_df = pd.DataFrame(
        tfidf_features.toarray(),
        columns=[f'tfidf_{i}' for i in range(tfidf_features.shape[1])]
    )

    # Numerical features
    numerical_features = job_data[[
        'experience_years_required',
        'salary_max',
        'num_skills_required',
        'description_length'
    ]]

    # Categorical features (one-hot encoded)
    categorical_features = pd.get_dummies(
        job_data[['job_type', 'location_type']],
        prefix=['type', 'loc']
    )

    # Combine all features
    features = pd.concat([
        tfidf_df,
        numerical_features,
        categorical_features
    ], axis=1)

    return features
```

Model Training:

```python
```

```python
import xgboost as xgb
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.metrics import classification_report
import mlflow
import mlflow.xgboost

def train_job_classifier(X_train, y_train, X_val, y_val):
    """Train XGBoost classifier for job categories"""

    # Start MLflow run
    with mlflow.start_run(run_name="job_category_classifier"):

        # Hyperparameter tuning
        param_grid = {
            'max_depth': [3, 5, 7],
            'learning_rate': [0.01, 0.1, 0.3],
            'n_estimators': [100, 200, 300],
            'subsample': [0.8, 0.9, 1.0],
            'colsample_bytree': [0.8, 0.9, 1.0]
        }

        xgb_model = xgb.XGBClassifier(
            objective='multi:softmax',
            num_class=len(np.unique(y_train)),
            random_state=42,
            eval_metric='mlogloss'
        )

        grid_search = GridSearchCV(
            xgb_model,
            param_grid,
            cv=5,
            scoring='f1_macro',
            n_jobs=-1,
            verbose=1
        )

        grid_search.fit(X_train, y_train)

        best_model = grid_search.best_estimator_

        # Evaluate
        val_predictions = best_model.predict(X_val)

        # Log parameters
        mlflow.log_params(grid_search.best_params_)
```

```python
    # Log metrics
    accuracy = accuracy_score(y_val, val_predictions)
    f1 = f1_score(y_val, val_predictions, average='macro')

    mlflow.log_metric("accuracy", accuracy)
    mlflow.log_metric("f1_macro", f1)

    # Log model
    mlflow.xgboost.log_model(best_model, "model")

    # Log classification report
    report = classification_report(y_val, val_predictions)
    mlflow.log_text(report, "classification_report.txt")

    print(f"Validation Accuracy: {accuracy:.4f}")
    print(f"Validation F1 (Macro): {f1:.4f}")

    return best_model
```

Component: Candidate-Job Matching Model

Technology: XGBoost, Neural Networks (PyTorch), MLflow **Model Type**: Regression (match score 0-100)

Feature Engineering:

```python
```

```python
def calculate_matching_features(candidate: Dict, job: Dict) -> pd.Series:
    """Calculate features for candidate-job matching"""

    features = {}

    # 1. Skill Matching
    candidate_skills = set(candidate['skills']['technical'])
    required_skills = set(job['required_skills'])

    features['skill_overlap_count'] = len(candidate_skills & required_skills)
    features['skill_overlap_ratio'] = (
        len(candidate_skills & required_skills) / len(required_skills)
        if required_skills else 0
    )
    features['skill_excess_count'] = len(candidate_skills - required_skills)

    # Skill embedding similarity (cosine similarity)
    features['skill_embedding_similarity'] = cosine_similarity(
        candidate['skill_embedding'].reshape(1, -1),
        job['skill_embedding'].reshape(1, -1)
    )[0][0]

    # 2. Experience Matching
    features['experience_years'] = candidate['total_experience_years']
    features['experience_gap'] = (
        candidate['total_experience_years'] - job['experience_required']
    )
    features['experience_match'] = min(
        candidate['total_experience_years'] / job['experience_required'], 1.0
    ) if job['experience_required'] > 0 else 1.0

    # 3. Education Matching
    education_levels = {
        'high_school': 1, 'associate': 2, 'bachelor': 3,
        'master': 4, 'phd': 5
    }

    candidate_edu_level = education_levels.get(
        candidate['highest_degree'].lower(), 0
    )
    required_edu_level = education_levels.get(
        job['education_required'].lower(), 0
    )

    features['education_level_candidate'] = candidate_edu_level
    features['education_level_required'] = required_edu_level
```

```
    features['education_match'] = (
        candidate_edu_level >= required_edu_level
    )

    # 4. Industry/Domain Match
    features['same_industry'] = (
        candidate['industry'] == job['industry']
    )

    # 5. Location Match
    features['location_distance_km'] = calculate_distance(
        candidate['location_coords'],
        job['location_coords']
    )
    features['same_city'] = (candidate['city'] == job['city'])

    # 6. Salary Expectations
    if candidate.get('expected_salary') and job.get('salary_max'):
        features['salary_gap'] = (
            candidate['expected_salary'] - job['salary_max']
        )
        features['salary_match'] = (
            candidate['expected_salary'] <= job['salary_max']
        )
    else:
        features['salary_gap'] = 0
        features['salary_match'] = True

    # 7. Emotion Profile (from video interview)
    if candidate.get('emotion_features'):
        features['confidence_level'] = candidate['emotion_features']['confidence_lev
        features['positive_ratio'] = candidate['emotion_features']['positive_negativ
        features['emotion_stability'] = 1 / (candidate['emotion_features']['emotion_

    # 8. Career Progression
    features['career_progression_score'] = calculate_career_progression(
        candidate['work_experience']
    )

    # 9. Recent Activity
    features['months_since_last_job'] = calculate_months_since_last_job(
        candidate['work_experience']
    )

    return pd.Series(features)
```

Neural Network Model:

```python
```

```python
import torch
import torch.nn as nn

class CandidateJobMatchingNN(nn.Module):
    def __init__(self, input_dim: int):
        super().__init__()

        self.network = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.Dropout(0.3),

            nn.Linear(256, 128),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Dropout(0.3),

            nn.Linear(128, 64),
            nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.Dropout(0.2),

            nn.Linear(64, 1),
            nn.Sigmoid()  # Output 0-1, scale to 0-100
        )

    def forward(self, x):
        return self.network(x) * 100  # Scale to 0-100

def train_matching_model(train_loader, val_loader, epochs=50):
    """Train matching neural network"""

    model = CandidateJobMatchingNN(input_dim=train_loader.dataset[0][0].shape[0])
    model = model.to(device)

    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode='min', patience=5
    )

    best_val_loss = float('inf')

    for epoch in range(epochs):
        # Training
```

```
        model.train()
        train_loss = 0
        for features, targets in train_loader:
            features, targets = features.to(device), targets.to(device)

            optimizer.zero_grad()
            outputs = model(features)
            loss = criterion(outputs.squeeze(), targets)
            loss.backward()
            optimizer.step()

            train_loss += loss.item()

        # Validation
        model.eval()
        val_loss = 0
        with torch.no_grad():
            for features, targets in val_loader:
                features, targets = features.to(device), targets.to(device)
                outputs = model(features)
                loss = criterion(outputs.squeeze(), targets)
                val_loss += loss.item()

        avg_train_loss = train_loss / len(train_loader)
        avg_val_loss = val_loss / len(val_loader)

        scheduler.step(avg_val_loss)

        if avg_val_loss < best_val_loss:
            best_val_loss = avg_val_loss
            torch.save(model.state_dict(), 'best_matching_model.pth')

        print(f'Epoch {epoch+1}/{epochs} - Train Loss: {avg_train_loss:.4f}, Val Los

    return model
```

2.6 API Layer

Component: FastAPI Backend

Technology: FastAPI 0.100+, SQLAlchemy, Pydantic, JWT

Project Structure:

```
backend/
├── app/
│   ├── api/
│   │   ├── v1/
│   │   │   ├── endpoints/
│   │   │   │   ├── auth.py
│   │   │   │   ├── candidates.py
│   │   │   │   ├── jobs.py
│   │   │   │   ├── matching.py
│   │   │   │   └── analytics.py
│   │   │   └── router.py
│   ├── core/
│   │   ├── config.py
│   │   ├── security.py
│   │   └── deps.py
│   ├── models/
│   │   ├── candidate.py
│   │   ├── job.py
│   │   └── application.py
│   ├── schemas/
│   │   ├── candidate.py
│   │   ├── job.py
│   │   └── matching.py
│   ├── services/
│   │   ├── cv_parser.py
│   │   ├── matching_engine.py
│   │   └── video_processor.py
│   └── main.py
```

API Endpoints:

```python
```

```python
from fastapi import FastAPI, HTTPException, Depends, UploadFile, File
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from pydantic import BaseModel
from typing import List, Optional

app = FastAPI(
    title="Recruitment Matching Platform API",
    version="1.0.0",
    description="AI-powered recruitment matching system"
)

# Authentication
@app.post("/api/v1/auth/login")
async def login(form_data: OAuth2PasswordRequestForm = Depends()):
    """User login endpoint"""
    user = authenticate_user(form_data.username, form_data.password)
    if not user:
        raise HTTPException(status_code=401, detail="Incorrect credentials")

    access_token = create_access_token(data={"sub": user.email})
    return {"access_token": access_token, "token_type": "bearer"}

# Candidate endpoints
@app.post("/api/v1/candidates/upload-cv")
async def upload_cv(
    file: UploadFile = File(...),
    current_user: User = Depends(get_current_user)
):
    """Upload and parse CV"""

    # Validate file type
    if file.content_type not in ["application/pdf", "application/vnd.openxmlformats-
        raise HTTPException(400, "Invalid file type. PDF or DOCX only.")

    # Save file
    file_path = await save_upload_file(file)

    # Parse CV asynchronously
    task = parse_cv_task.delay(file_path, current_user.id)

    return {
        "message": "CV uploaded successfully",
        "task_id": task.id,
        "status": "processing"
    }
```

```python
@app.get("/api/v1/candidates/{candidate_id}")
async def get_candidate(
    candidate_id: int,
    current_user: User = Depends(get_current_user)
):
    """Get candidate profile"""
    candidate = get_candidate_by_id(candidate_id)
    if not candidate:
        raise HTTPException(404, "Candidate not found")
    return candidate

# Job endpoints
@app.post("/api/v1/jobs")
async def create_job(
    job: JobCreate,
    current_user: User = Depends(get_current_recruiter)
):
    """Create new job posting"""
    job_db = create_job_posting(job, current_user.id)
    return job_db

@app.get("/api/v1/jobs/search")
async def search_jobs(
    q: Optional[str] = None,
    category: Optional[str] = None,
    location: Optional[str] = None,
    skip: int = 0,
    limit: int = 20
):
    """Search job postings"""
    jobs = search_jobs_query(q, category, location, skip, limit)
    return jobs

# Matching endpoints
@app.post("/api/v1/matching/find-matches")
async def find_matches(
    candidate_id: int,
    top_n: int = 10,
    current_user: User = Depends(get_current_user)
):
    """Find top matching jobs for candidate"""

    # Get candidate profile
    candidate = get_candidate_by_id(candidate_id)

    # Get all active jobs
    active_jobs = get_active_jobs()
```

```python
    # Calculate match scores
    matches = []
    for job in active_jobs:
        score = calculate_match_score(candidate, job)
        matches.append({
            "job_id": job.id,
            "job_title": job.title,
            "company": job.company,
            "match_score": score,
            "matching_skills": get_matching_skills(candidate, job)
        })

    # Sort by score and return top N
    matches.sort(key=lambda x: x['match_score'], reverse=True)
    return matches[:top_n]

@app.post("/api/v1/matching/video-analysis")
async def analyze_interview_video(
    file: UploadFile = File(...),
    candidate_id: int,
    current_user: User = Depends(get_current_user)
):
    """Upload and analyze interview video"""

    # Validate video file
    if not file.content_type.startswith("video/"):
        raise HTTPException(400, "Invalid file type. Video only.")

    # Save video
    video_path = await save_upload_file(file)

    # Process video asynchronously
    task = process_video_task.delay(video_path, candidate_id)

    return {
        "message": "Video uploaded successfully",
        "task_id": task.id,
        "status": "processing"
    }

# Analytics endpoints
@app.get("/api/v1/analytics/matching-performance")
async def get_matching_performance(
    start_date: datetime,
    end_date: datetime,
    current_user: User = Depends(get_current_admin)
```

```
):
    """Get matching algorithm performance metrics"""

    metrics = calculate_performance_metrics(start_date, end_date)
    return metrics
```

2.7 Database Schema

PostgreSQL Schema:

```sql
```

```sql
-- Users table
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(50) NOT NULL CHECK (role IN ('candidate', 'recruiter', 'admin')),
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Candidates table
CREATE TABLE candidates (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    phone VARCHAR(20),
    location VARCHAR(255),
    city VARCHAR(100),
    country VARCHAR(100),
    linkedin_url VARCHAR(255),
    github_url VARCHAR(255),
    total_experience_years INTEGER,
    highest_degree VARCHAR(100),
    industry VARCHAR(100),
    expected_salary INTEGER,
    cv_file_path VARCHAR(500),
    video_file_path VARCHAR(500),
    skill_embedding VECTOR(384),  -- pgvector extension
    cv_embedding VECTOR(384),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Skills table
CREATE TABLE skills (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) UNIQUE NOT NULL,
    category VARCHAR(50) CHECK (category IN ('technical', 'soft', 'tool', 'language'
    normalized_name VARCHAR(100)
);

-- Candidate skills (many-to-many)
CREATE TABLE candidate_skills (
    candidate_id INTEGER REFERENCES candidates(id) ON DELETE CASCADE,
```

```sql
    skill_id INTEGER REFERENCES skills(id) ON DELETE CASCADE,
    proficiency_level VARCHAR(50) CHECK (proficiency_level IN ('beginner', 'intermed
    years_experience DECIMAL(4,1),
    PRIMARY KEY (candidate_id, skill_id)
);

-- Education table
CREATE TABLE education (
    id SERIAL PRIMARY KEY,
    candidate_id INTEGER REFERENCES candidates(id) ON DELETE CASCADE,
    degree VARCHAR(100),
    field_of_study VARCHAR(100),
    institution VARCHAR(255),
    start_date DATE,
    end_date DATE,
    gpa DECIMAL(3,2)
);

-- Work experience table
CREATE TABLE work_experience (
    id SERIAL PRIMARY KEY,
    candidate_id INTEGER REFERENCES candidates(id) ON DELETE CASCADE,
    job_title VARCHAR(255),
    company VARCHAR(255),
    location VARCHAR(255),
    start_date DATE,
    end_date DATE,
    is_current BOOLEAN DEFAULT false,
    description TEXT,
    responsibilities TEXT[]
);

-- Companies table
CREATE TABLE companies (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) UNIQUE NOT NULL,
    industry VARCHAR(100),
    size VARCHAR(50),
    website VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Jobs table
CREATE TABLE jobs (
    id SERIAL PRIMARY KEY,
    company_id INTEGER REFERENCES companies(id),
    recruiter_id INTEGER REFERENCES users(id),
```

```sql
    title VARCHAR(255) NOT NULL,
    description TEXT NOT NULL,
    category VARCHAR(100),
    job_type VARCHAR(50) CHECK (job_type IN ('full-time', 'part-time', 'contract', '
    location VARCHAR(255),
    city VARCHAR(100),
    country VARCHAR(100),
    location_type VARCHAR(50) CHECK (location_type IN ('on-site', 'remote', 'hybrid'
    experience_required INTEGER,
    education_required VARCHAR(100),
    salary_min INTEGER,
    salary_max INTEGER,
    num_positions INTEGER DEFAULT 1,
    status VARCHAR(50) DEFAULT 'active' CHECK (status IN ('draft', 'active', 'closed
    skill_embedding VECTOR(384),
    posted_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    closing_date TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Job skills (many-to-many)
CREATE TABLE job_skills (
    job_id INTEGER REFERENCES jobs(id) ON DELETE CASCADE,
    skill_id INTEGER REFERENCES skills(id) ON DELETE CASCADE,
    is_required BOOLEAN DEFAULT true,
    importance_level INTEGER CHECK (importance_level BETWEEN 1 AND 5),
    PRIMARY KEY (job_id, skill_id)
);

-- Applications table
CREATE TABLE applications (
    id SERIAL PRIMARY KEY,
    job_id INTEGER REFERENCES jobs(id) ON DELETE CASCADE,
    candidate_id INTEGER REFERENCES candidates(id) ON DELETE CASCADE,
    match_score DECIMAL(5,2),
    status VARCHAR(50) DEFAULT 'submitted' CHECK (status IN ('submitted', 'screening
    applied_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    recruiter_notes TEXT,
    UNIQUE(job_id, candidate_id)
);

-- Emotion analysis table
CREATE TABLE emotion_analysis (
    id SERIAL PRIMARY KEY,
    candidate_id INTEGER REFERENCES candidates(id) ON DELETE CASCADE,
```

```sql
    video_file_path VARCHAR(500),
    dominant_emotion VARCHAR(50),
    confidence_level DECIMAL(4,3),
    positive_ratio DECIMAL(4,3),
    emotion_stability DECIMAL(4,3),
    happy_mean DECIMAL(4,3),
    sad_mean DECIMAL(4,3),
    angry_mean DECIMAL(4,3),
    fear_mean DECIMAL(4,3),
    surprise_mean DECIMAL(4,3),
    neutral_mean DECIMAL(4,3),
    analyzed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Indexes for performance
CREATE INDEX idx_candidates_user_id ON candidates(user_id);
CREATE INDEX idx_jobs_company_id ON jobs(company_id);
CREATE INDEX idx_jobs_status ON jobs(status);
CREATE INDEX idx_applications_job_id ON applications(job_id);
CREATE INDEX idx_applications_candidate_id ON applications(candidate_id);
CREATE INDEX idx_candidate_skills_candidate_id ON candidate_skills(candidate_id);
CREATE INDEX idx_job_skills_job_id ON job_skills(job_id);

-- Full-text search indexes
CREATE INDEX idx_jobs_title_fts ON jobs USING GIN(to_tsvector('english', title));
CREATE INDEX idx_jobs_description_fts ON jobs USING GIN(to_tsvector('english', descr
```

---

3. Deployment Architecture

Production Infrastructure (AWS/GCP):

```
┌──────────────────────────────────────────────────────────┐
│                    CloudFront CDN                        │
│                (Frontend Distribution)                   │
└──────────────────────┬───────────────────────────────────┘
                       │
┌──────────────────────┴───────────────────────────────────┐
│                Application Load Balancer                 │
│                  (SSL Termination)                       │
└──────────────────────┬───────────────────────────────────┘
                       │
           ┌───────────┴──────────┐
           │                      │
┌──────────▼──────────┐     ┌─────────▼─────────┐
│  Frontend           │     │   Backend         │
│  (S3 Static)        │     │   (ECS/EKS)       │
│                     │     │   Auto-scaling    │
└─────────────────────┘     │   2-10 instances│
                            └──────────┬────────┘
                                       │
                            ┌──────────┴──────────┐
                            │                     │
                  ┌─────────▼─────────┐   ┌─────────▼─────────┐
                  │ PostgreSQL        │   │  Redis            │
                  │ (RDS Multi-AZ)│   │ (ElastiCache)│
                  └───────────────────┘   └───────────────────┘
                            │
                  ┌─────────▼─────────┐
                  │ S3 Buckets        │
                  │ - CVs             │
                  │ - Videos          │
                  │ - ML Models       │
                  └───────────────────┘
```

This architecture document provides the technical foundation. The next document will cover monitoring, observability, and operational considerations.